Agentic Workbench UX Research

Date: 2026-05-09 Scope: Research basis for the SaaStoAgent operator workbench and future generated REST tool, execution, QA, and learning surfaces.

Product Implication

SaaStoAgent should present an operating workbench, not a generic chatbot with side panels. The user should always be able to answer:

- What can SaaStoAgent do now?
- What is Corpus doing now?
- What setup or approval is blocking progress?
- What evidence, source data, tool state, or trace supports the output?
- How can the user correct, take over, approve, or save a learning?

The practical product loop remains:

```
Describe the job -> See readiness -> Approve or adjust plan -> Let Corpus act -> Inspect evidence -> Give feedback or save learning
```

Patterns To Preserve

1. Chat Is The Intent Spine, Not The Whole Product

Microsoft's agent UX guidance distinguishes agents from typical chatbots because agents include instructions, knowledge, actions, skills, memory, and tools. It also emphasizes that agentic systems need transparency, control, privacy, visible status, and access to knowledge/tools rather than relying on text chat alone.

For SaaStoAgent:

- Keep the central thread as the place where the user describes the job and directs Corpus.
- Put readiness, capability state, action approvals, traces, and evidence into structured UI around the thread.
- Do not turn every future slice into another chat prompt or another full-page route.

Sources:

- Microsoft Design, "UX design for agents": https://microsoft.design/articles/ux-design-for-agents/
- Microsoft HAX, "Make clear what the system can do": https://www.microsoft.com/en-us/haxtoolkit/guideline/make-clear-what-the-system-can-do/

2. Capability Boundaries Must Be Visible

Microsoft HAX Guideline 1 says users need to understand what an AI system can do and what it is designed for; unclear task/domain expectations can cause disappointment, abandonment, or harm.

For SaaStoAgent:

- Keep the capability rail stateful: Ready, Needs setup, Locked, Running, Needs approval, Has findings.

- Explain locked or setup-needed capabilities at the point of use.
- Let generated REST tools, entities, QA, and learnings register capability state before they become visible as ready.

Source:

- Microsoft HAX, "Make clear what the system can do": https://www.microsoft.com/en-us/haxtoolkit/guideline/make-clear-what-the-system-can-do/

3. Progressive Disclosure Is The Power Model

Microsoft's agent UX principles describe agent interaction as gradual and increasingly powerful over time. The same guidance calls for visible agent status, transparent/customizable tools and connections, and familiar UI controls where possible.

For SaaStoAgent:

- Default view should stay quiet: product header, status strip, chat, next best action, collapsed evidence.
- Power surfaces should be progressive: context lens, evidence drawer, tool traces, request/response previews, approval history, learning candidates.
- New surfaces should not overwhelm first-run users or hide capability from power users.

Source:

- Microsoft Design, "UX design for agents": https://microsoft.design/articles/ux-design-for-agents/

4. Failure And Manual Recovery Are First-Class UX

Google PAIR mental-model and graceful-failure guidance emphasizes that failure should not create dead ends, and that users should have a non-AI/manual way to complete work when the AI cannot proceed. PAIR also recommends feedback opportunities around errors and correct outputs.

For SaaStoAgent:

- Missing API, missing generated tools, auth-required actions, stream failure, unsupported artifacts, and risky execution should all produce recoverable states.
- Evidence and feedback should live near output, traces, and failure states.
- Feedback should not automatically change behavior; it should become a governed learning candidate until QA validates it.

Sources:

- Google PAIR, "Mental Models": https://pair.withgoogle.com/guidebook-v2/chapter/mental-models/
- Google PAIR, "Feedback + Control": https://pair.withgoogle.com/guidebook-v2/chapters/feedback-controls/
- Google PAIR, "Errors + Graceful Failure": https://pair.withgoogle.com/chapter/errors-failing/

5. Approvals And Tool Scope Belong In The Core Interaction

OpenAI's workspace-agent materials emphasize that users decide what tools/data an agent can use, what actions it can take, and when approval is required. OpenAI's help material also calls out write-action approvals,

risky workflows, and least-privilege controls for agents using app or connector credentials.

For SaaStoAgent:

- The visible autonomy ladder is the right future surface, but backend approval gates remain authoritative.
- Generated REST tools should expose risk category, auth requirement, read/write posture, and approval requirement before execution.
- Sensitive write actions should default to ask/approve behavior until a governed policy model exists.

Sources:

- OpenAI, "Introducing workspace agents in ChatGPT": https://openai.com/index/introducing-workspace-agents-in-chatgpt/
- OpenAI Help Center, "ChatGPT Workspace Agents for Enterprise and Business": https://help.openai.com/en/articles/20001143-chatgpt-workspace-agents-for-enterprise-and-business

6. Human-In-The-Loop Agents Need Co-Planning, Guards, And Memory

Magentic-UI frames human-in-the-loop agentic systems as a way to combine human oversight/control with AI efficiency, and names co-planning, co-tasking, multi-tasking, action guards, and long-term memory as interaction mechanisms. Apple research on computer-use agents maps the UX design space and highlights that developers need to consider different user needs and scenarios when designing agent interactions.

For SaaStoAgent:

- Execution plans should be inspectable and adjustable before REST action execution.
- Risky or irreversible actions need action guards and visible approvals.
- Sessions, memory, QA findings, and saved learnings should be inspectable as workspace capability surfaces.
- Corpus should remember validated workspace knowledge, not every transient chat correction.

Sources:

- Magentic-UI, "Towards Human-in-the-loop Agentic Systems": https://arxiv.org/abs/2507.22358
- Apple Machine Learning Research, "Mapping the Design Space of User Experience for Computer Use Agents": https://machinelearning.apple.com/research/mapping

SaaStoAgent Baseline Requirements

Every future capability slice should define these before it is treated as product-ready:

- User-visible capability state.
- Primary backend-owned action.
- Empty state.
- Failure/recovery state.
- Evidence surface.
- Approval or autonomy posture if actions can execute.
- Feedback capture path.
- Test scenarios for first-run, everyday, power-user, failure, and mobile behavior.

Current Implementation Mapping

- `frontend/src/components/OperatorGateway.tsx` owns the unified shell and runtime bridge.
- `frontend/src/components/operator/OperatorWorkbench.tsx` owns status strip, rail, action dock, context lens, evidence drawer, and autonomy ladder.
- `frontend/src/lib/operatorExperience.ts` owns the registry-driven capability model.
- `frontend/src/lib/entryGraph.ts` owns product/operator display constants and legacy workspace display-name cleanup.
- `decisions/ADR-006-operator-workbench-extensibility-contract.md` records the accepted interface contract.

Next Research/Application Target

Apply this research directly to Slice 2B:

- generated REST tool inspection
- tool risk and auth posture display
- chat-to-tool candidate selection
- execution-plan artifact
- approval-request artifact
- trace-summary artifact
- learning-candidate artifact